Linear velocity calibration

This lesson uses the Raspberry Pi as an example. For Raspberry Pi and Jetson-Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this lesson in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01.Robot Configuration and Operation Guide] -- [5.Enter Docker (For JETSON Nano and RPi 5)]**. For Orin and RDK boards, simply open the terminal and enter the commands mentioned in this lesson.

1. Program Description

Run the program and adjust the parameters in the dynamic parameter adjuster to calibrate the car's linear speed. To visually demonstrate the linear speed, instruct the car to move straight forward for 1 meter and observe its actual distance to see if it is within the error range.

2. Starting the Program

2.1. Startup Commands

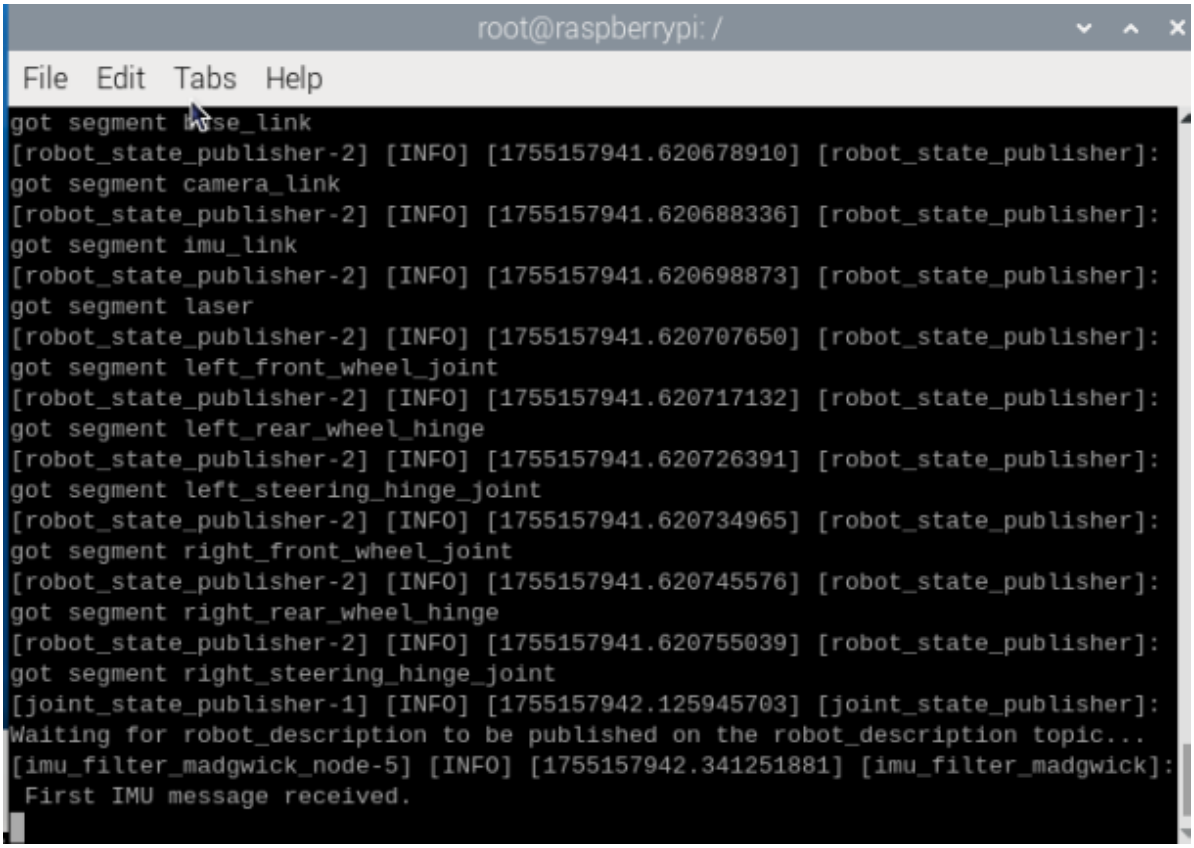
For the Raspberry Pi 5 and jetson nano controller, you must first enter the Docker container. For the Orin and RDK controller, this is not necessary.

Entering the Docker container (see the Docker course for steps: 4. Docker Startup Script).

All subsequent commands must be executed within the same Docker container (see the Docker course for steps: 3. Docker Commit and Multi-Terminal Entry).

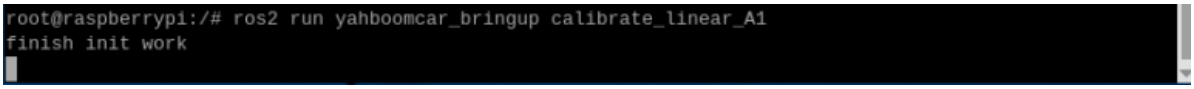
Start the chassis data: Enter the command in the terminal.

```
ros2 launch yahboomcar_bringup yahboomcar_bringup_A1_launch.py
```

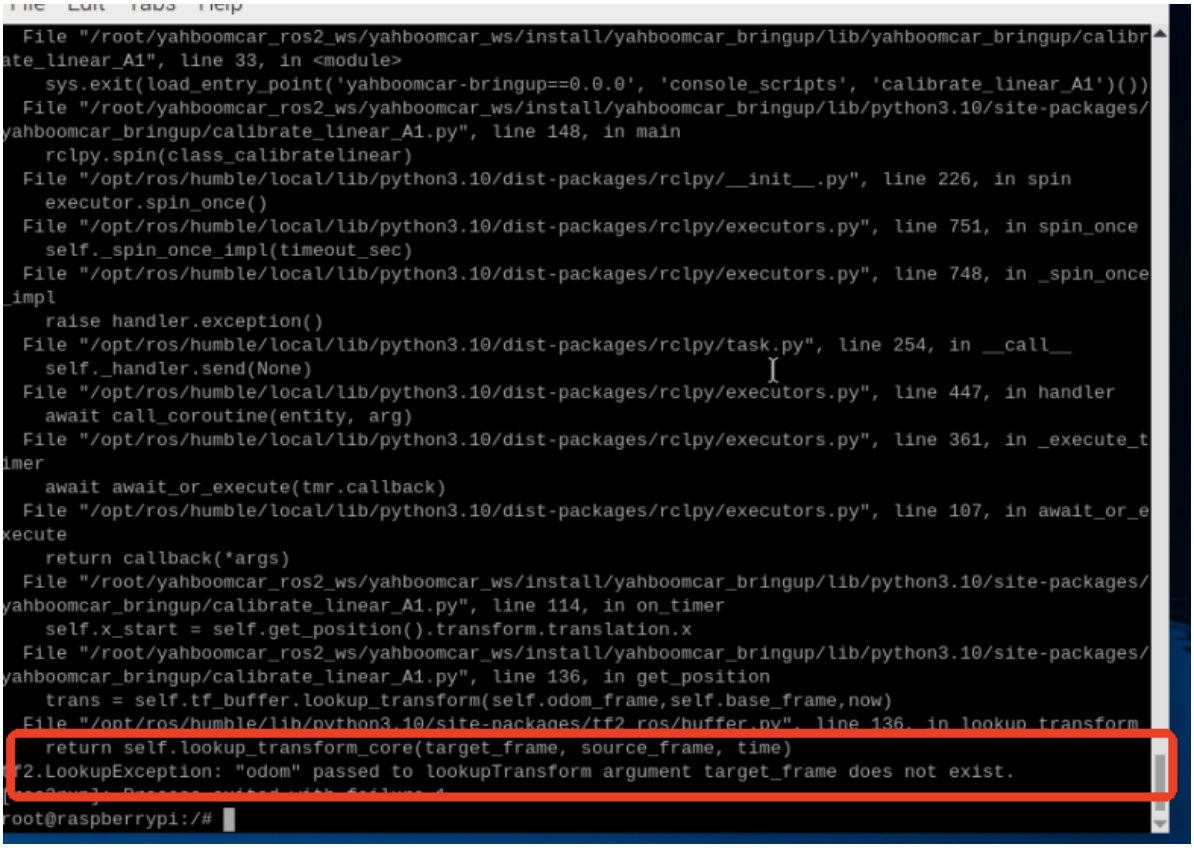



```
ros2 run yahboomcar_bringup calibrate_linear_A1
```

The following image is displayed successfully.



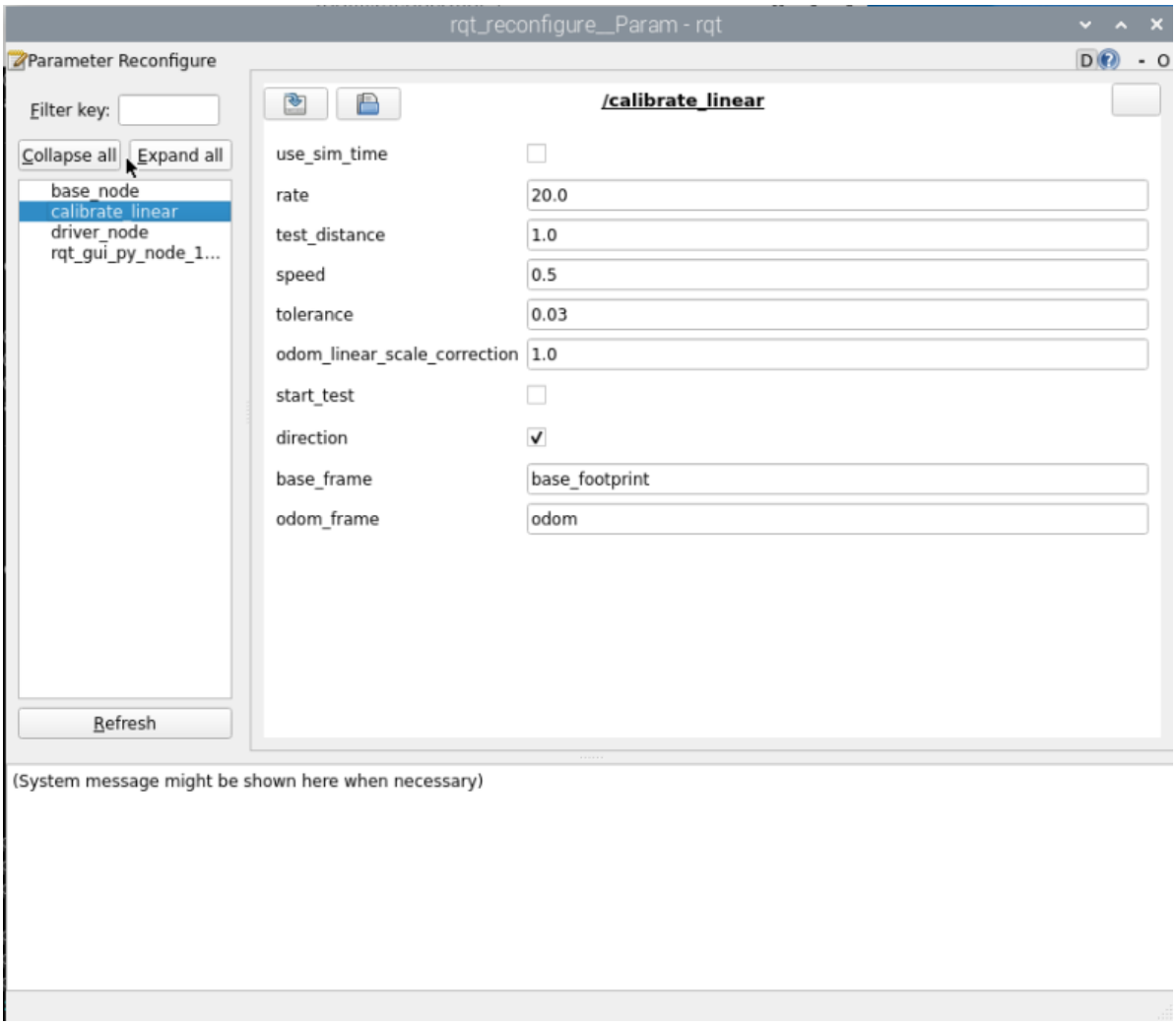
If the following error message appears during runtime indicating that no TF transformations were found, press **Ctrl+C** to exit the program and run it again.



Open the dynamic parameter adjuster and run the following command in the terminal:

```
ros2 run rqt_reconfigure rqt_reconfigure
```

Click the **calibrate_linear** node in the node options on the left:



Note: The above nodes may not appear when you first open the application. Click Refresh to see all nodes. The **calibrate_linear** node displayed is the node for calibrating linear velocity.

The rqt interface parameters are described as follows:

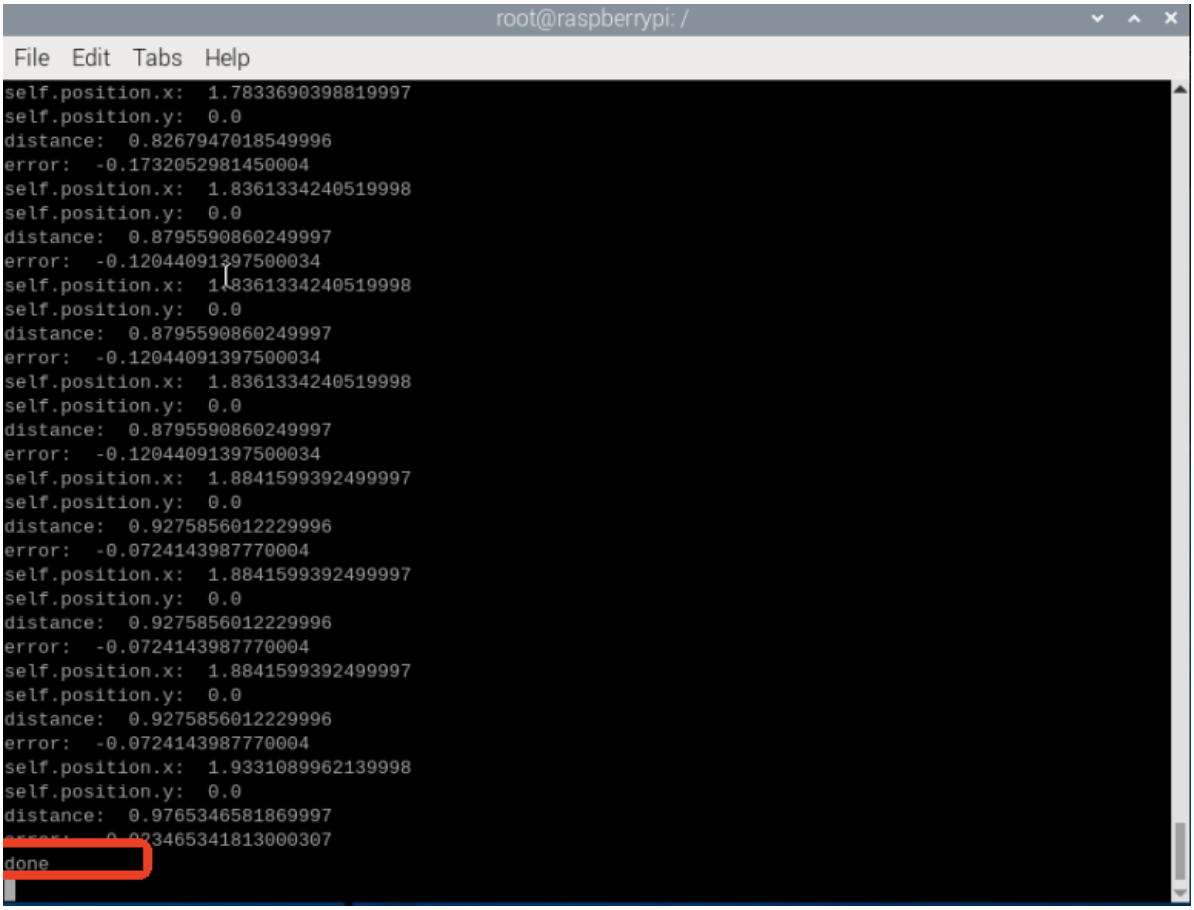
- test_distance: Calibration test distance. Here, the test is a 1-meter forward movement.
- speed: Linear speed.
- tolerance: Error tolerance.
- odom_linear_scale_correction: Linear speed scaling factor. If the test results are unsatisfactory, adjust this value.
- start_test: Test switch.
- direction: Can be ignored. This value is used for the McLennan robot. Modifying it allows you to calibrate the linear speed of left and right movements.
- base_frame: The name of the base coordinate system.
- odom_frame: The name of the odometry coordinate system.

2.2. Start Calibration

In the rqt_reconfigure interface, select the calibrate_linear node (click **Refresh** if it doesn't appear).

Select a reference of known length on the ground (a tape measure, tile, etc.). Change **test_distance** to the actual test distance (1 meter is used as an example). Click the **start_test** box to begin calibration.

After clicking start_test, calibration begins. The robot will monitor the TF transformations of base_footprint and odom, calculate the theoretical distance traveled, and when the error is less than tolerance, issue a stop command and the terminal will print "done." If the actual distance traveled is less than 1 meter, increase the **odom_linear_scale_correction** parameter appropriately. After making these changes, click a blank space, click start_test again, reset the start_test value, and click start_test again to complete the calibration. Modifying other parameters is similar; click a blank space to write the modified parameters. Record the final calibrated **odom_linear_scale_correction** parameter.



After testing, remember the value of odom_linear_scale_correction and modify it to the linear_scale_x parameter in yahboomcar_bringup_A1_launch.py.

Path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/launch/yahboomcar_bringup_A1_launch.py
```

base_node_A1.cpp | calibrate_linear_A1.py | yahboomcar_bringup_A1_launch.py

yahboomcar_ros2_ws > yahboomcar_ws > src > yahboomcar_bringup > launch > yahboomcar_bringup_A1_launch.py

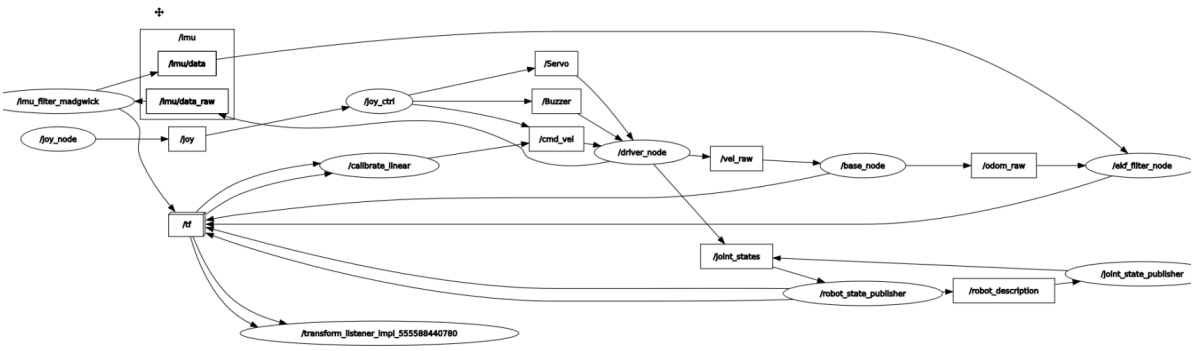
print 第？项，共4项

```python
def generate_launch_description():
    rviz_node = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen',
        arguments=['-d', LaunchConfiguration('rvizconfig')],
    )

    imu_filter_config = os.path.join(
        get_package_share_directory('yahboomcar_bringup'),
        'param',
        'imu_filter_param.yaml'
    )

    driver_node = Node(
        package='yahboomcar_bringup',
        executable='Ackman_driver_A1',
    )

    base_node = Node(
        package='yahboomcar_base_node',
        executable='base_node_A1',
        # 当使用ekf融合时，该tf有ekf发布
        parameters=[{
            'pub_odom_tf': LaunchConfiguration('pub_odom_tf'),
            'linear_scale_x': 1.0,
            'linear_scale_y': 1.0,
        }]
    )

    imu_filter_node = Node(
        package='imu_filter_madgwick',
        executable='imu_filter_madgwick_node',
        parameters=[imu_filter_config]
    )

    ekf_node = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
```

3. View the Node Relationship Graph

Open a terminal and enter the command:

```
ros2 run rqt_graph rqt_graph
```



In the above node relationship graph:

- The imu_filter node filters the chassis' raw IMU data (/imu/data) and publishes the filtered data (/imu/data).
- The ekf_filter_node node subscribes to the chassis' raw odometry data (/odom_raw) and filtered IMU data (/imu/data), fuses the data, and publishes it to the topic (/odom).

- The calibrate_linear node monitors the TF transformation from odom to base_footprint and publishes to the topic (/cmd_vel) to control the robot's chassis movement.

4. Core Source Code Analysis

This program primarily utilizes TF monitoring of coordinate transformations. By monitoring the coordinate transformation between base_footprint and odom, the robot can determine "how far I've traveled" or "how many degrees I've turned."

Code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/yahboomcar_bringup/cal
ibrate_angular_A1.py
```

Among them, the implementation of monitoring tf coordinate transformation is the get_position method in the CalibrateLinear class:

```
def get_position(self):
     try:
        now = rclpy.time.Time()
        transform = self.tf_buffer.lookup_transform(
            self.base_frame,
            self.odom_frame,
            now,
            timeout=rclpy.duration.Duration(seconds=1.0))
        return transform

     except (LookupException, ConnectivityException, ExtrapolationException):
        self.get_logger().info('transform not ready')
        raise
```

The on_timer method (timer callback function) in the CalibrateLinear class is used to determine the displacement of the robot chassis and control its movement:

```
def on_timer(self):
    move_cmd = Twist()
    #self.get_param()
    self.start_test =
self.get_parameter('start_test').get_parameter_value().bool_value
    self.odom_linear_scale_correction =
self.get_parameter('odom_linear_scale_correction').get_parameter_value().double_
value
    self.rate = self.get_parameter('rate').get_parameter_value().double_value
    self.test_distance =
self.get_parameter('test_distance').get_parameter_value().double_value
    self.direction =
self.get_parameter('direction').get_parameter_value().double_value
    self.tolerance =
self.get_parameter('tolerance').get_parameter_value().double_value
    self.speed = self.get_parameter('speed').get_parameter_value().double_value
    if self.start_test:
        self.position.x = self.get_position().transform.translation.x
        self.position.y = self.get_position().transform.translation.y
        print("self.position.x: ",self.position.x)
        print("self.position.y: ",self.position.y)
        distance = sqrt(pow((self.position.x - self.x_start), 2) +
```

```
                            pow((self.position.y - self.y_start), 2))
        distance *= self.odom_linear_scale_correction
        print("distance: ",distance)
        error = distance - self.test_distance
        print("error: ",error)
        #start = time()
        if not self.start_test or abs(error) < self.tolerance:
            self.start_test  =
rclpy.parameter.Parameter('start_test',rclpy.Parameter.Type.BOOL,False)
            all_new_parameters = [self.start_test]
            self.set_parameters(all_new_parameters)

            print("done")
        else:
            move_cmd.linear.x = copysign(self.speed, -1 * error)
            '''if self.direction:
                print("x")
                move_cmd.linear.x = copysign(self.speed, -1 * error)
            else:
                move_cmd.linear.y = copysign(self.speed, -1 * error)
                print("y")'''
        self.cmd_vel.publish(move_cmd)

    else:
        self.x_start = self.get_position().transform.translation.x
        self.y_start = self.get_position().transform.translation.y

        self.cmd_vel.publish(Twist())
```